



Skill: pallets_itsdangerous

➤ "Usa este skill cuando necesites firmar criptográficamente datos, generar tokens seguros, serializar objetos con firma, implementar expiración de tokens, rotar claves secretas, o manejar datos en entornos no confiables.

ItsDangerous - Firma y Autenticación de Datos

Cuándo usar este skill

Este skill debe usarse cuando se requiera:

- Firmar datos para garantizar integridad y autenticidad en entornos no confiables (cookies, tokens de sesión, enlaces de verificación).
- Serializar objetos Python (JSON) y firmarlos para su transmisión segura.
- Implementar tokens con expiración temporal (enlaces de restablecimiento de contraseña, tokens de confirmación de email).
- Rotar claves secretas sin invalidar tokens existentes.
- Migrar algoritmos de firma manteniendo compatibilidad con tokens antiguos.
- Generar tokens seguros para URLs (Base64 URL-safe).

Estructura del proyecto

El repositorio de ItsDangerous se organiza en los siguientes módulos principales:

- `itsdangerous/__init__.py` - Punto de entrada, importa todas las clases públicas.
- `itsdangerous/signer.py` - Clases base `Signer` y `NoneAlgorithm`.
- `itsdangerous/serializer.py` - `Serializer` genérico.
- `itsdangerous/timed.py` - `TimestampSigner` y `TimedSerializer`.
- `itsdangerous/url_safe.py` - `URLSafeSerializer` y `URLSafeTimedSerializer`.
- `itsdangerous/exc.py` - Jerarquía de excepciones.
- `itsdangerous/encoding.py` - Utilidades de codificación (base64, bytes).

API principal

Firmas básicas

Signer - Firma bytes y verifica la firma.

```
1 class Signer:
2     def __init__(
3         self,
4         secret_key: str | bytes | Iterable[str] | Iterable[bytes],
5         salt: str | bytes | None = b"itsdangerous.Signer",
6         sep: str | bytes = b".",
7         key_derivation: str | None = None,
8         digest_method: Any | None = None,
9         algorithm: SigningAlgorithm | None = None,
10    )
11    def sign(self, value: str | bytes) -> bytes
12    def unsign(self, signed_value: str | bytes) -> bytes # lanza
13        BadSignature
14    def validate(self, signed_value: str | bytes) -> bool
```

SigningAlgorithm - Clase base para algoritmos de firma.

```
1 class SigningAlgorithm:
2     def get_signature(self, key: bytes, value: bytes) -> bytes
3     def verify_signature(self, key: bytes, value: bytes, sig: bytes) ->
4         bool
```

HMACAlgorithm - Algoritmo HMAC con hash configurable.

```
1 class HMACAlgorithm(SigningAlgorithm):
2     default_digest_method = staticmethod(_lazy_sha1) # hashlib.sha1
3     def __init__(self, digest_method=None)
```

NoneAlgorithm - Algoritmo de firma vacía (solo para pruebas).

```
1 class NoneAlgorithm(SigningAlgorithm):
2     def get_signature(self, key: bytes, value: bytes) -> bytes
3     def verify_signature(self, key: bytes, value: bytes, sig: bytes) ->
4         bool
```

Serialización

Serializer - Serializa objetos (por defecto JSON) y los firma.

```

1 class Serializer(Generic[_TSerialized]):
2     default_serializer = json
3     default_signer = Signer
4     default_fallback_signers = []
5
6     def __init__(
7         self,
8         secret_key: ...,
9         salt: str | bytes | None = b"itsdangerous",
10        serializer: ... = None,
11        serializer_kwargs: dict | None = None,
12        signer: type[Signer] | None = None,
13        signer_kwargs: dict | None = None,
14        fallback_signers: list | None = None,
15    )
16    def dumps(self, obj: Any, salt: ... = None) -> _TSerialized
17    def loads(self, s: str | bytes, salt: ... = None, **kwargs) -> Any
18        # lanza BadSignature
19    def loads_unsafe(self, s: str | bytes, salt: ... = None) -> tuple[
20        bool, Any]
21    def load_payload(self, payload: bytes, serializer: ... = None) ->
22        Any
23    def dump_payload(self, obj: Any) -> bytes

```

Firmas con timestamp

TimestampSigner - Añade timestamp a la firma.

```

1 class TimestampSigner(Signer):
2     def get_timestamp() -> int
3     def timestamp_to_datetime(ts: int) -> datetime
4     def sign(self, value: str | bytes) -> bytes
5     def unsign(self, signed_value: str | bytes, max_age: int | None =
6         None, return_timestamp: bool = False) -> bytes | tuple[bytes,
7         datetime]
8     def validate(self, signed_value: str | bytes, max_age: int | None =
9         None) -> bool

```

TimedSerializer - Serializa y añade timestamp.

```

1 class TimedSerializer(Serializer):
2     def loads(self, s: str | bytes, max_age: int | None = None,
3         return_timestamp: bool = False, salt: ... = None) -> Any
4     def loads_unsafe(self, s: str | bytes, max_age: int | None = None,
5         salt: ... = None) -> tuple[bool, Any]

```

Serializadores URL-safe

URLSafeSerializer - Serializa en formato seguro para URLs.

```
1 class URLSafeSerializer(URLSafeSerializerMixin, Serializer[str]):
2     def load_payload(self, payload: bytes, ...) -> Any
3     def dump_payload(self, obj: Any) -> bytes
```

URLSafeTimedSerializer - Combina URL-safe con timestamp.

```
1 class URLSafeTimedSerializer(URLSafeSerializerMixin, TimedSerializer[
    str]):
2     pass
```

Utilidades de encoding

```
1 def want_bytes(s: str | bytes, encoding="utf-8", errors="strict") ->
    bytes
2 def base64_encode(string: str | bytes) -> bytes
3 def base64_decode(string: str | bytes) -> bytes
```

Excepciones

```
1 class BadData(Exception)           # base
2 class BadSignature(BadData)         # firma no coincide (payload
    disponible)
3 class BadTimeSignature(BadSignature) # timestamp inválido (date_signed
    disponible)
4 class SignatureExpired(BadTimeSignature) # expirado
5 class BadHeader(BadSignature)        # cabecera malformada
6 class BadPayload(BadData)            # payload no válido (original_error
    disponible)
```

Patrones de uso

1. Firma básica con serializador URL-safe

```
1 from itsdangerous import URLSafeSerializer
2
3 s = URLSafeSerializer("secret-key", salt="auth")
4 token = s.dumps({"id": 5, "name": "itsdangerous"})
5 data = s.loads(token)
```

2. Tokens con expiración temporal

```
1 from itsdangerous import TimestampSigner
2
3 s = TimestampSigner("secret-key")
4 signed = s.sign("foo")
5 s.unsign(signed, max_age=5) # lanza SignatureExpired si > 5 segundos
```

3. Rotación de claves secretas

```
1 SECRET_KEYS = ["old-key", "new-key"]
2 s = Serializer(SECRET_KEYS)
3 token = s.dumps({"id": 42}) # firma con "new-key"
4
5 # Rotar clave
6 SECRET_KEYS.append("newer-key")
7 del SECRET_KEYS[0]
8 s = Serializer(SECRET_KEYS)
9 s.loads(token) # válido, se prueba con todas las claves
```

4. Migración de algoritmo con fallback signers

```
1 import hashlib
2
3 s = Serializer(
4     "secret-key",
5     signer_kwargs={"digest_method": hashlib.sha512},
6     fallback_signers=[{"digest_method": hashlib.sha1}]
7 )
```

5. Manejo de errores con inspección de payload

```
1 from itsdangerous.exc import BadSignature, BadData
2
3 try:
4     data = s.loads(token)
5 except BadSignature as e:
6     if e.payload is not None:
7         try:
8             decoded = s.load_payload(e.payload)
9         except BadData:
10             pass # payload corrupto
```

6. Carga insegura para debugging

```
1 sig_ok, payload = s.loads_unsafe(data)
```

Configuración

Parámetros principales

- `secret_key`: Clave secreta (bytes aleatorios). Puede ser lista para rotación.
- `salt`: Distingue contextos de firma. No necesita ser secreto.
- `key_derivation`: Método para derivar clave ("django-concat", "concat", "hmac", "none").
- `digest_method`: Función hash para HMAC (por defecto `hashlib.sha1`).
- `serializer`: Objeto con `dumps/loads` (por defecto `json`).
- `fallback_signers`: Lista de configuraciones para verificar firmas antiguas.
- `sep`: Separador entre valor y firma (por defecto ". ").
- `max_age`: Edad máxima en segundos para tokens temporales.
- `return_timestamp`: Si es `True`, `unsign/loads` retorna también el timestamp.

Ejemplos de código reutilizable

Generar token de confirmación de email

```
1 from itsdangerous import URLSafeTimedSerializer
2
3 serializer = URLSafeTimedSerializer("secret-key", salt="email-confirm")
4 token = serializer.dumps({"user_id": 123})
5 # Enviar token por email...
```

Verificar token con expiración

```
1 from itsdangerous.exc import SignatureExpired, BadSignature
2
3 try:
4     data = serializer.loads(token, max_age=3600) # 1 hora
5 except SignatureExpired:
6     print("El token ha expirado")
7 except BadSignature:
8     print("Token inválido")
```

Rotar claves en aplicación Flask

```
1 from itsdangerous import URLSafeTimedSerializer
2
3 class TokenManager:
4     def __init__(self, secret_keys):
5         self.secret_keys = secret_keys
6         self.serializer = URLSafeTimedSerializer(secret_keys)
7
8     def rotate_key(self, new_key):
9         self.secret_keys.insert(0, new_key)
10        self.serializer = URLSafeTimedSerializer(self.secret_keys)
```

Glosario

- **Firma criptográfica:** Proceso de generar un código (HMAC) que verifica la integridad y autenticidad de los datos.
- **HMAC:** Hash-based Message Authentication Code, algoritmo que combina una clave secreta con un hash.
- **Salt:** Valor aleatorio que se añade a la clave para derivar claves diferentes en distintos contextos.
- **Key derivation:** Proceso de generar una clave criptográfica a partir de una clave maestra y un salt.
- **Timestamp:** Marca de tiempo Unix que indica cuándo se firmó el dato.
- **URL-safe:** Codificación Base64 que usa caracteres seguros para URLs (- y _ en lugar de + y /).
- **Fallback signers:** Configuraciones de firmantes antiguos que permiten verificar tokens firmados con algoritmos o claves anteriores.
- **Rotación de claves:** Práctica de cambiar periódicamente las claves secretas manteniendo las antiguas para validar tokens existentes.

Prerrequisitos

- Python 3.8 o superior.
- No tiene dependencias externas (solo stdlib).
- Instalación: `pip install itsdangerous`

api

API de ItsDangerous

Módulo `itsdangerous.signer`

Signer

Firma bytes y verifica la firma usando HMAC.

```
1 class Signer:
2     def __init__(
3         self,
4         secret_key: str | bytes | Iterable[str] | Iterable[bytes],
5         salt: str | bytes | None = b"itsdangerous.Signer",
6         sep: str | bytes = b".",
7         key_derivation: str | None = None,
8         digest_method: Any | None = None,
9         algorithm: SigningAlgorithm | None = None,
10    )
```

Parámetros:

- `secret_key`: Clave secreta. Puede ser una cadena, bytes, o iterable de claves para rotación.
- `salt`: Valor para derivar claves diferentes en distintos contextos. No necesita ser secreto.
- `sep`: Separador entre el valor y la firma (por defecto `"."`).
- `key_derivation`: Método de derivación de clave (`"concat"`, `"django-concat"`, `"hmac"`, `"none"`).
- `digest_method`: Función hash para HMAC (por defecto `hashlib.sha1`).
- `algorithm`: Instancia de `SigningAlgorithm` (por defecto `HMACAlgorithm`).

Métodos:

```
1 def sign(self, value: str | bytes) -> bytes
```

Firma un valor y retorna `valor.separador.firma`.

```
1 def unsign(self, signed_value: str | bytes) -> bytes
```

Verifica la firma y retorna el valor original. Lanza `BadSignature` si la firma es inválida.

```
1 def validate(self, signed_value: str | bytes) -> bool
```

Retorna True si la firma es válida, False en caso contrario.

Ejemplo:

```
1 from itsdangerous import Signer
2
3 s = Signer("secret-key")
4 signed = s.sign("hello")
5 # b'hello.XXXXX'
6 original = s.unsign(signed)
7 # b'hello'
```

SigningAlgorithm

Clase base abstracta para algoritmos de firma.

```
1 class SigningAlgorithm:
2     def get_signature(self, key: bytes, value: bytes) -> bytes
3     def verify_signature(self, key: bytes, value: bytes, sig: bytes) ->
4         bool
```

HMACAlgorithm

Implementación HMAC del algoritmo de firma.

```
1 class HMACAlgorithm(SigningAlgorithm):
2     default_digest_method = staticmethod(_lazy_sha1)
3
4     def __init__(self, digest_method=None)
```

Parámetros:

- `digest_method`: Función hash (por defecto `hashlib.sha1`).

NoneAlgorithm

Algoritmo de firma vacía que no genera firma real. Solo para pruebas.

```
1 class NoneAlgorithm(SigningAlgorithm):
2     def get_signature(self, key: bytes, value: bytes) -> bytes
3     def verify_signature(self, key: bytes, value: bytes, sig: bytes) ->
4         bool
```

Módulo `itsdangerous.serializer`**Serializer**

Serializa objetos Python (por defecto JSON) y los firma.

```
1 class Serializer(Generic[_TSerialized]):
2     default_serializer = json
3     default_signer = Signer
4     default_fallback_signers = []
5
6     def __init__(
7         self,
8         secret_key: str | bytes | Iterable[str] | Iterable[bytes],
9         salt: str | bytes | None = b"itsdangerous",
10        serializer: Any = None,
11        serializer_kwargs: dict | None = None,
12        signer: type[Signer] | None = None,
13        signer_kwargs: dict | None = None,
14        fallback_signers: list | None = None,
15    )
```

Parámetros:

- `secret_key`: Clave secreta (igual que en `Signer`).
- `salt`: Salt para derivación de clave.
- `serializer`: Módulo u objeto con `dumps/loads` (por defecto `json`).
- `serializer_kwargs`: Argumentos adicionales para el serializador.
- `signer`: Clase de firmante a usar (por defecto `Signer`).
- `signer_kwargs`: Argumentos para el constructor del firmante.
- `fallback_signers`: Lista de diccionarios con `kwargs` para firmantes de respaldo.

Métodos:

```
1 def dumps(self, obj: Any, salt: str | bytes | None = None) ->
    _TSerialized
```

Serializa `obj` a JSON, lo firma y retorna el resultado.

```
1 def loads(self, s: str | bytes, salt: str | bytes | None = None, **
    kwargs) -> Any
```

Verifica la firma y deserializa el payload. Lanza `BadSignature` si falla.

```
1 def loads_unsafe(self, s: str | bytes, salt: str | bytes | None = None)
    -> tuple[bool, Any]
```

Retorna (éxito, `payload`) sin lanzar excepciones.

```
1 def load_payload(self, payload: bytes, serializer: Any = None) -> Any
```

Deserializa un payload ya verificado.

```
1 def dump_payload(self, obj: Any) -> bytes
```

Serializa un objeto a payload sin firmar.

Ejemplo:

```
1 from itsdangerous import Serializer
2
3 s = Serializer("secret-key")
4 token = s.dumps({"user": "alice"})
5 # b'{"user":"alice"}.XXXXX'
6 data = s.loads(token)
7 # {"user": "alice"}
```

Módulo `itsdangerous.timed`

TimestampSigner

Firmante que añade timestamp a la firma.

```
1 class TimestampSigner(Signer):
2     def get_timestamp() -> int
3     def timestamp_to_datetime(ts: int) -> datetime
4     def sign(self, value: str | bytes) -> bytes
5     def unsign(
6         self,
7         signed_value: str | bytes,
8         max_age: int | None = None,
9         return_timestamp: bool = False
10    ) -> bytes | tuple[bytes, datetime]
11    def validate(self, signed_value: str | bytes, max_age: int | None =
        None) -> bool
```

Métodos:

```
1 def get_timestamp() -> int
```

Retorna la marca de tiempo Unix actual.

```
1 def timestamp_to_datetime(ts: int) -> datetime
```

Convierte timestamp Unix a datetime UTC.

```
1 def sign(self, value: str | bytes) -> bytes
```

Firma el valor con timestamp: valor.sep.timestamp.sep.firma.

```
1 def unsign(
2     self,
3     signed_value: str | bytes,
4     max_age: int | None = None,
5     return_timestamp: bool = False
6 ) -> bytes | tuple[bytes, datetime]
```

Verifica firma y timestamp. Si max_age se supera, lanza SignatureExpired. Si return_timestamp=True, retorna (valor, datetime).

```
1 def validate(self, signed_value: str | bytes, max_age: int | None =
    None) -> bool
```

Retorna True si la firma es válida y no ha expirado.

Ejemplo:

```
1 from itsdangerous import TimestampSigner
2
3 s = TimestampSigner("secret-key")
4 signed = s.sign("foo")
5 # b'foo.1234567890.XXXXX'
6 original = s.unsign(signed, max_age=3600)
7 # b'foo'
```

TimedSerializer

Serializador que añade timestamp.

```
1 class TimedSerializer(Serializer):
2     def loads(
3         self,
4         s: str | bytes,
5         max_age: int | None = None,
6         return_timestamp: bool = False,
7         salt: str | bytes | None = None
8     ) -> Any
```

```

9     def loads_unsafe(
10         self,
11         s: str | bytes,
12         max_age: int | None = None,
13         salt: str | bytes | None = None
14     ) -> tuple[bool, Any]

```

Ejemplo:

```

1  from itsdangerous import TimedSerializer
2
3  s = TimedSerializer("secret-key")
4  token = s.dumps({"user": 42})
5  data = s.loads(token, max_age=300)

```

Módulo `itsdangerous.url_safe``URLSafeSerializerMixin`

Mixin que añade codificación Base64 URL-safe y compresión zlib.

```

1  class URLSafeSerializerMixin:
2      def load_payload(self, payload: bytes, *args, **kwargs) -> Any
3      def dump_payload(self, obj: Any) -> bytes

```

`URLSafeSerializer`

Serializador con salida URL-safe.

```

1  class URLSafeSerializer(URLSafeSerializerMixin, Serializer[str]):
2      pass

```

`URLSafeTimedSerializer`

Serializador URL-safe con timestamp.

```

1  class URLSafeTimedSerializer(URLSafeSerializerMixin, TimedSerializer[
2      str]):
3      pass

```

Ejemplo:

```

1  from itsdangerous import URLSafeTimedSerializer
2

```

```
3 s = URLSafeTimedSerializer("secret-key")
4 token = s.dumps({"session_id": "abc"})
5 # 'eyJzZXNzaW9uX2lkIjoIYWJjIn0.XXXXX.XXXXX'
6 data = s.loads(token, max_age=86400)
```

Módulo itsdangerous.encoding

Funciones de utilidad

```
1 def want_bytes(s: str | bytes, encoding="utf-8", errors="strict") ->
    bytes
```

Convierte cadena a bytes si es necesario.

```
1 def base64_encode(string: str | bytes) -> bytes
```

Codifica en Base64 URL-safe.

```
1 def base64_decode(string: str | bytes) -> bytes
```

Decodifica Base64 URL-safe.

Módulo itsdangerous.exc

Jerarquía de excepciones

```
1 class BadData(Exception):
2     """Excepción base para datos inválidos."""
3
4 class BadPayload(BadData):
5     """Payload no pudo ser deserializado."""
6     def __init__(self, message, original_error=None)
7
8 class BadSignature(BadData):
9     """Firma no coincide."""
10    def __init__(self, message, payload=None, original_error=None)
11
12 class BadTimeSignature(BadSignature):
13     """Timestamp inválido o malformado."""
14    def __init__(self, message, payload=None, date_signed=None,
15                  original_error=None)
16
17 class SignatureExpired(BadTimeSignature):
```

```
17     """Firma expirada por max_age."""
18     def __init__(self, message, payload=None, date_signed=None,
19                   original_error=None):
20
21 class BadHeader(BadSignature):
22     """Cabecera del payload malformada."""
```

configuracion

Configuración de ItsDangerous

Parámetros de configuración

Signer y derivados

Parámetro	Tipo	Default	Descripción
secret_key	'str \	bytes \	Iterable[str] \
salt	'str \	bytes \	None'
sep	'str \	bytes'	b". "
key_derivation	'str \	None'	None (equivale a "concat")
digest_method	'callable \	None'	hashlib.sha1
algorithm	'SigningAlgorithm \	None'	HMACAlgorithm()

Serializer y derivados

Parámetro	Tipo	Default	Descripción
secret_key	'str \	bytes \	Iterable[str] \
salt	'str \	bytes \	None'
serializer	'module \	object'	json
serializer_kwargs	'dict \	None'	None
signer	'type[Signer] \	None'	Signer
signer_kwargs	'dict \	None'	None
fallback_signers	'list \	None'	[]

TimestampSigner y TimedSerializer

Parámetro	Tipo	Default	Descripción
max_age	'int\	None'	None
return_timestamp	bool	False	Si True, <code>unsign()</code> / <code>loads()</code> retornan también el timestamp como datetime UTC

`URLSafeSerializer` y `URLSafeTimedSerializer`

Estos serializadores aplican automáticamente:

- Codificación Base64 URL-safe (caracteres `-` y `_` en lugar de `+` y `/`).
- Compresión zlib si el payload comprimido es menor que el original (indicado con prefijo `b" . "`).

No tienen parámetros de configuración adicionales.

Métodos de derivación de clave (`key_derivation`)

- `"concat"` (default implícito): Concatena `salt + b"signer" + secret_key`.
- `"django-concat"`: Concatena `salt + b"signer"` y usa `secret_key` como clave directa (compatible con Django).
- `"hmac"`: Deriva la clave usando HMAC con el `salt` como mensaje y `secret_key` como clave.
- `"none"`: No deriva, usa `secret_key` directamente.

Algoritmos de firma disponibles

- `HMACAlgorithm`: Algoritmo por defecto. Usa HMAC con la función hash especificada en `digest_method`.
- `NoneAlgorithm`: No genera firma real. Solo para pruebas, nunca en producción.

Variables de entorno

ItsDangerous no requiere variables de entorno. La configuración se realiza programáticamente.

Mejores prácticas de configuración

1. **Usar salts diferentes para cada contexto:** Evita que un token generado para un propósito sea válido en otro.

```
1 session_s = URLSafeSerializer(secret, salt="session")
2 email_s = URLSafeSerializer(secret, salt="email")
```

1. **Rotar claves periódicamente:** Mantén una lista de claves, añadiendo nuevas al inicio y eliminando antiguas.

```
1 SECRET_KEYS = ["current-key", "previous-key"]
```

1. **Usar `fallback_signers` para migrar algoritmos:** Permite cambiar el algoritmo de hash sin invalidar tokens existentes.

```
1 Serializer(  
2     secret,  
3     signer_kwargs={"digest_method": hashlib.sha256},  
4     fallback_signers=[{"digest_method": hashlib.sha1}]  
5 )
```

1. **Elegir el serializador adecuado:**
 2. `Serializer`: Para tokens que no van en URLs.
 3. `URLSafeSerializer`: Para tokens en URLs o cookies.
 4. `TimedSerializer` / `URLSafeTimedSerializer`: Cuando se necesita expiración.
1. **Configurar `max_age` según el caso de uso:**
 2. Enlaces de restablecimiento de contraseña: 1 hora (3600).
 3. Tokens de sesión: 1 día (86400).
 4. Enlaces de confirmación de email: 7 días (604800).

ejemplos

Ejemplos de código de ItsDangerous

Ejemplo 1: Firma básica con Signer

```
1 from itsdangerous import Signer  
2  
3 s = Signer("secret-key")
```

```
4 signed = s.sign(b"hello")
5 print(signed) # b'hello.XXXXX'
6
7 original = s.unsign(signed)
8 print(original) # b'hello'
9
10 # Validar sin excepción
11 if s.validate(signed):
12     print("Firma válida")
```

Ejemplo 2: Serialización JSON con Serializer

```
1 from itsdangerous import Serializer
2
3 s = Serializer("secret-key")
4 token = s.dumps({"user": "alice", "id": 42})
5 print(token) # b'{"user": "alice", "id": 42}.XXXXX'
6
7 data = s.loads(token)
8 print(data) # {"user": "alice", "id": 42}
```

Ejemplo 3: Token con expiración usando TimestampSigner

```
1 from itsdangerous import TimestampSigner
2 from itsdangerous.exc import SignatureExpired
3
4 s = TimestampSigner("secret-key")
5 signed = s.sign("datos-sensibles")
6 print(signed) # b'datos-sensibles.1234567890.XXXXX'
7
8 try:
9     original = s.unsign(signed, max_age=5) # 5 segundos
10 except SignatureExpired:
11     print("Token expirado")
```

Ejemplo 4: Serializador URL-safe con tiempo

```
1 from itsdangerous import URLSafeTimedSerializer
2
3 s = URLSafeTimedSerializer("secret-key", salt="email-confirm")
4 token = s.dumps({"user_id": 123})
5 print(token)  # 'eyJ1c2VyX2lkIjoxMjN9.XXXXX.XXXXX'
6
7 data = s.loads(token, max_age=3600)  # 1 hora
8 print(data)  # {"user_id": 123}
```

Ejemplo 5: Rotación de claves

```
1 from itsdangerous import Serializer
2
3 # Claves iniciales
4 SECRET_KEYS = ["old-key", "new-key"]
5 s = Serializer(SECRET_KEYS)
6
7 # Firmar con "new-key"
8 token = s.dumps({"id": 42})
9
10 # Rotar clave
11 SECRET_KEYS.insert(0, "newer-key")
12 SECRET_KEYS.pop()  # eliminar "old-key"
13 s = Serializer(SECRET_KEYS)
14
15 # Verificar token antiguo
16 data = s.loads(token)  # funciona con "new-key"
17 print(data)  # {"id": 42}
```

Ejemplo 6: Migración de algoritmo con fallback

```
1 import hashlib
2 from itsdangerous import Serializer
3
4 s = Serializer(
5     "secret-key",
6     signer_kwargs={"digest_method": hashlib.sha512},
7     fallback_signers=[{"digest_method": hashlib.sha1}]
8 )
9
10 # Firmar con SHA512
```

```
11 token = s.dumps({"data": "test"})
12
13 # Verificar (funciona con ambos algoritmos)
14 data = s.loads(token)
15 print(data) # {"data": "test"}
```

Ejemplo 7: Manejo de errores con payload

```
1 from itsdangerous import Serializer
2 from itsdangerous.exc import BadSignature, BadPayload
3
4 s = Serializer("secret-key")
5 token = s.dumps({"user": "bob"})
6
7 # Simular token corrupto
8 try:
9     data = s.loads(token + b"tampered")
10 except BadSignature as e:
11     print(f"Firma inválida: {e}")
12     if e.payload:
13         try:
14             decoded = s.load_payload(e.payload)
15             print(f"Payload: {decoded}")
16         except BadPayload:
17             print("Payload corrupto")
```

Ejemplo 8: Carga insegura

```
1 from itsdangerous import Serializer
2
3 s = Serializer("secret-key")
4 token = s.dumps({"user": "alice"})
5
6 sig_ok, payload = s.loads_unsafe(token)
7 print(f"Válido: {sig_ok}, Payload: {payload}")
8 # Válido: True, Payload: {"user": "alice"}
9
10 # Con token inválido
11 sig_ok, payload = s.loads_unsafe("token-invalid")
12 print(f"Válido: {sig_ok}, Payload: {payload}")
13 # Válido: False, Payload: None
```

Ejemplo 9: Uso de salt para diferentes contextos

```
1 from itsdangerous import URLSafeSerializer
2
3 # Diferentes salts para diferentes propósitos
4 session_serializer = URLSafeSerializer("secret-key", salt="session")
5 email_serializer = URLSafeSerializer("secret-key", salt="email")
6
7 session_token = session_serializer.dumps({"user_id": 123})
8 email_token = email_serializer.dumps({"action": "verify"})
9
10 # No se puede usar un token en otro contexto
11 try:
12     session_serializer.loads(email_token) # Lanza BadSignature
13 except BadSignature:
14     print("Token de email no válido para sesión")
```

Ejemplo 10: Obtener timestamp de un token

```
1 from itsdangerous import TimestampSigner
2
3 s = TimestampSigner("secret-key")
4 signed = s.sign("data")
5
6 # Obtener valor y timestamp
7 value, timestamp = s.unsign(signed, return_timestamp=True)
8 print(f"Valor: {value}")
9 print(f"Firmado: {timestamp}") # datetime UTC
```

glosario

Glosario de ItsDangerous

Términos técnicos

Firma criptográfica

Proceso de generar un código de autenticación (MAC) que permite verificar que los datos no han sido alterados y provienen de una fuente confiable. ItsDangerous usa HMAC para este propósito.

HMAC (Hash-based Message Authentication Code)

Algoritmo que combina una clave secreta con una función hash (como SHA1, SHA256) para generar un código de autenticación. Verifica tanto la integridad como la autenticidad de los datos.

Clave secreta (secret_key)

Valor criptográfico (bytes aleatorios) usado para firmar y verificar datos. Debe mantenerse en secreto. Puede ser una lista para soportar rotación de claves.

Salt

Valor aleatorio o único que se añade a la clave secreta durante la derivación para generar claves diferentes en distintos contextos. No necesita ser secreto. Ejemplo: usar salt="session" para cookies de sesión y salt="email" para tokens de email.

Derivación de clave (key derivation)

Proceso de generar una clave criptográfica a partir de una clave maestra y un salt. ItsDangerous soporta varios métodos: concat, django-concat, hmac, none.

Separador (sep)

Carácter o bytes que separan el valor original de la firma en el resultado firmado. Por defecto es " . ".

Payload

Los datos originales (serializados) que se firman. En `Serializer`, es el resultado de `json.dumps()`. En caso de firma inválida, el payload puede estar disponible para inspección.

Timestamp

Marca de tiempo Unix (segundos desde 1970-01-01 00:00:00 UTC) que indica cuándo se firmó el dato. Usado por `TimestampSigner` para validar expiración.

max_age

Edad máxima permitida en segundos para un token con timestamp. Si el token es más antiguo, se lanza `SignatureExpired`.

URL-safe

Codificación que usa caracteres seguros para incluir en URLs sin necesidad de percent-encoding. ItsDangerous usa Base64 modificada: - en lugar de +, _ en lugar de /, y omite el padding =.

Base64

Esquema de codificación que representa datos binarios en texto ASCII. ItsDangerous usa Base64 URL-safe para tokens que van en URLs.

Compresión zlib

Algoritmo de compresión sin pérdida. Los serializadores URL-safe comprimen el payload con zlib si el resultado es menor, anteponiendo `b" . "` para indicar compresión.

Fallback signers

Lista de configuraciones de firmantes antiguos que permiten verificar tokens firmados con algoritmos o configuraciones anteriores. Facilita la migración de algoritmos sin invalidar tokens existentes.

Rotación de claves

Práctica de seguridad que consiste en cambiar periódicamente las claves secretas. ItsDangerous soporta rotación manteniendo una lista de claves: la primera se usa para firmar, todas se usan para verificar.

BadSignature

Excepción lanzada cuando la firma de un token no coincide con la esperada. Indica que los datos pueden haber sido manipulados o que la clave/salt es incorrecta.

BadTimeSignature

Excepción lanzada cuando el timestamp de un token está ausente o malformado. Hereda de `BadSignature`.

SignatureExpired

Excepción lanzada cuando un token con timestamp ha excedido el `max_age` especificado. Hereda de `BadTimeSignature`.

BadPayload

Excepción lanzada cuando el payload no puede ser deserializado (ej. JSON inválido).

loads_unsafe

Método que retorna una tupla (`éxito`, `payload`) sin lanzar excepciones. Útil para debugging o cuando se necesita inspeccionar el contenido sin manejar errores.

Serializador

Componente que convierte objetos Python a un formato de bytes (ej. JSON) y viceversa. En ItsDangerous, el serializador por defecto es `json`.

Signer

Componente que firma bytes y verifica firmas. Es la clase base para todos los firmantes en ItsDangerous.

Algoritmo de firma (SigningAlgorithm)

Clase abstracta que define cómo se genera y verifica una firma. Implementaciones: `HMCAAlgorithm` (por defecto) y `NoneAlgorithm` (pruebas).

patrones

Patrones de uso de ItsDangerous

1. Firma y verificación básica

Problema: Necesitas enviar datos a un cliente (ej. cookie) y verificar que no han sido manipulados al recibirlos de vuelta.

Solución: Usar `Signer` para firmar los datos.

Flujo:

1. Crear instancia de `Signer` con clave secreta y salt único para el contexto.
2. Firmar los datos con `sign()`.
3. Enviar datos firmados al cliente.
4. Al recibir, verificar con `unsign()` o `validate()`.
5. Si `unsign()` lanza `BadSignature`, los datos fueron manipulados.

```
1 from itsdangerous import Signer
2
3 # 1. Configurar firmante
4 signer = Signer("clave-secreta", salt="cookie-session")
5
6 # 2. Firmar datos
7 signed_data = signer.sign("user_id=123")
8
9 # 3. Enviar al cliente (ej. en cookie)
10
11 # 4. Verificar al recibir
12 try:
13     original = signer.unsign(signed_data)
14     print(f"Datos válidos: {original}")
15 except BadSignature:
16     print("Datos manipulados")
```

2. Tokens con expiración temporal

Problema: Generar tokens que expiran después de un tiempo (enlaces de restablecimiento de contraseña, confirmación de email).

Solución: Usar `TimestampSigner` o `TimedSerializer` con `max_age`.

Flujo: 1. Crear `TimestampSigner` o `TimedSerializer`.

1. Firmar/serializar datos con `sign()` o `dumps()`.
2. Enviar token al usuario.
3. Al recibir, verificar con `unsign()` o `loads()` especificando `max_age`.
4. Capturar `SignatureExpired` para tokens expirados.

```
1 from itsdangerous import TimestampSigner
2 from itsdangerous.exc import SignatureExpired, BadTimeSignature
3
4 # 1. Configurar
5 signer = TimestampSigner("clave-secreta", salt="reset-password")
6
7 # 2. Generar token
8 token = signer.sign("user_id=123")
9
10 # 3. Enviar por email...
11
12 # 4. Verificar con expiración de 1 hora
13 try:
14     user_id = signer.unsign(token, max_age=3600)
15     print(f"Token válido: {user_id}")
16 except SignatureExpired:
17     print("Token expirado, solicitar nuevo")
18 except BadTimeSignature:
19     print("Token inválido")
```

3. Serialización de objetos con firma

Problema: Necesitas enviar estructuras de datos complejas (dicts, listas) de forma segura.

Solución: Usar `Serializer` con JSON.

Flujo:

1. Crear `Serializer` con clave secreta.
2. Serializar objeto con `dumps()`.
3. Transmitir el token.
4. Deserializar con `loads()`.
5. Manejar `BadSignature` para datos inválidos.

```
1 from itsdangerous import Serializer
2
3 # 1. Configurar
4 serializer = Serializer("clave-secreta", salt="session")
5
6 # 2. Serializar y firmar
7 token = serializer.dumps({"user_id": 123, "role": "admin"})
8
9 # 3. Enviar token...
10
11 # 4. Deserializar
12 try:
13     data = serializer.loads(token)
14     print(f"Datos: {data}")
15 except BadSignature:
16     print("Token inválido")
```

4. Rotación de claves secretas

Problema: Necesitas cambiar la clave secreta periódicamente sin invalidar tokens existentes.

Solución: Pasar una lista de claves a Signer/Serializer. La primera clave se usa para firmar, todas se prueban para verificar.

Flujo:

1. Inicializar con lista de claves (nueva primero).
2. Firmar con la primera clave.
3. Al rotar, añadir nueva clave al inicio y eliminar la más antigua.
4. Tokens antiguos se verifican con claves anteriores.

```
1 from itsdangerous import Serializer
2
3 # 1. Claves iniciales
4 SECRET_KEYS = ["old-key", "new-key"]
5 serializer = Serializer(SECRET_KEYS)
6
7 # 2. Firmar con "new-key"
8 token = serializer.dumps({"id": 42})
9
10 # 3. Rotar clave
11 SECRET_KEYS.insert(0, "newer-key")
12 SECRET_KEYS.pop() # eliminar "old-key"
13 serializer = Serializer(SECRET_KEYS)
```

```
14
15 # 4. Verificar token antiguo (funciona con "new-key")
16 data = serializer.loads(token) # válido
```

5. Migración de algoritmo de firma

Problema: Cambiar el algoritmo de hash (ej. SHA1 a SHA512) manteniendo compatibilidad con tokens antiguos.

Solución: Usar `fallback_signers` para definir configuraciones de firmantes antiguos.

Flujo:

1. Configurar `Serializer` con nuevo algoritmo en `signer_kwargs`.
2. Pasar `fallback_signers` con configuraciones antiguas.
3. Tokens nuevos se firman con algoritmo nuevo.
4. Tokens antiguos se verifican con fallback signers.

```
1 import hashlib
2 from itsdangerous import Serializer
3
4 # 1. Configurar con nuevo algoritmo y fallback
5 serializer = Serializer(
6     "secret-key",
7     signer_kwargs={"digest_method": hashlib.sha512},
8     fallback_signers=[{"digest_method": hashlib.sha1}]
9 )
10
11 # 2. Firmar nuevo token (SHA512)
12 token = serializer.dumps({"data": "test"})
13
14 # 3. Token antiguo (SHA1) también se verifica
15 old_token = "..." # firmado con SHA1
16 data = serializer.loads(old_token) # válido por fallback
```

6. Tokens URL-safe para enlaces

Problema: Generar tokens que puedan incluirse en URLs sin caracteres problemáticos.

Solución: Usar `URLSafeSerializer` o `URLSafeTimedSerializer`.

Flujo:

1. Crear serializador URL-safe.

2. Serializar y firmar.
3. Incluir token en URL.
4. Al recibir, deserializar.

```
1 from itsdangerous import URLSafeTimedSerializer
2
3 # 1. Configurar
4 serializer = URLSafeTimedSerializer("secret-key", salt="email-verify")
5
6 # 2. Generar token
7 token = serializer.dumps({"user_id": 123})
8 # 'eyJ1c2VyX2lkIjoxMjN9.XXXXX.XXXXX'
9
10 # 3. Construir URL
11 url = f"https://example.com/verify?token={token}"
12
13 # 4. Verificar al recibir
14 try:
15     data = serializer.loads(token, max_age=86400)
16     print(f"Usuario verificado: {data['user_id']}")
17 except SignatureExpired:
18     print("Enlace expirado")
```

7. Manejo robusto de errores con inspección de payload

Problema: Necesitas diagnosticar por qué falló la verificación (firma inválida vs payload corrupto).

Solución: Capturar excepciones específicas y usar payload para inspeccionar.

Flujo:

1. Intentar loads().
2. Capturar BadSignature y verificar e.payload.
3. Si hay payload, intentar deserializarlo con load_payload().
4. Capturar BadPayload si el payload está corrupto.

```
1 from itsdangerous import Serializer
2 from itsdangerous.exc import BadSignature, BadPayload
3
4 serializer = Serializer("secret-key")
5
6 try:
7     data = serializer.loads(token)
```

```
8 except BadSignature as e:
9     if e.payload is not None:
10         try:
11             # Intentar recuperar payload aunque firma sea inválida
12             decoded = serializer.load_payload(e.payload)
13             print(f"Payload recuperado: {decoded}")
14         except BadPayload:
15             print("Payload corrupto")
16     else:
17         print("Token sin payload")
```

8. Carga insegura para debugging

Problema: Necesitas inspeccionar el contenido de un token sin que lance excepciones.

Solución: Usar `loads_unsafe()` que retorna (éxito, payload).

```
1 from itsdangerous import Serializer
2
3 serializer = Serializer("secret-key")
4
5 sig_ok, payload = serializer.loads_unsafe(token)
6 if sig_ok:
7     print(f"Token válido: {payload}")
8 else:
9     print(f"Token inválido, payload: {payload}")
```

prerrequisitos

Prerrequisitos de ItsDangerous

Requisitos del sistema

- **Python:** 3.8 o superior.
- **Sistema operativo:** Cualquier sistema que soporte Python (Windows, macOS, Linux).

Dependencias

ItsDangerous **no tiene dependencias externas**. Solo utiliza módulos de la biblioteca estándar de Python:

- `hashlib` - Para funciones hash (SHA1, SHA256, SHA512, etc.).
- `hmac` - Para HMAC (Hash-based Message Authentication Code).
- `json` - Para serialización JSON (serializador por defecto).
- `base64` - Para codificación Base64 URL-safe.
- `zlib` - Para compresión de payloads en serializadores URL-safe.
- `datetime` - Para manejo de timestamps.
- `time` - Para obtener timestamps Unix.
- `typing` - Para anotaciones de tipo.

Instalación

Desde PyPI (recomendado)

```
1 pip install itsdangerous
```

Desde el repositorio

```
1 git clone https://github.com/pallets/itsdangerous.git
2 cd itsdangerous
3 pip install -e .
```

Verificar instalación

```
1 import itsdangerous
2 print(itsdangerous.__version__)
```

Compatibilidad

- Compatible con Python 3.8, 3.9, 3.10, 3.11, 3.12.
- No tiene dependencias de sistema operativo.
- Puede usarse en cualquier framework web (Flask, Django, FastAPI, etc.) o aplicación Python.

Testing

Para ejecutar los tests del proyecto:

```
1 pip install -e "[dev]"
2 pytest
```

Licencia

BSD-3-Clause License. Ver archivo LICENSE en el repositorio.